



# Factory Method

Human Computer Interaction Research  
University of Nevada, Reno



# Pattern Categories

- Structural Patterns
  - » decorator
- Behavioral Patterns
  - » observer
  - » strategy
- Creational Patterns
  - » factory method

# example

Duck `duck`;

```
if (picnic) {  
    duck.bill = "yellow";  
}  
else if (hunting) {  
    duck.bill = "wooden";  
}  
else if (inBathTub) {  
    duck.bill = "rubber";  
}
```

additional  
attributes?

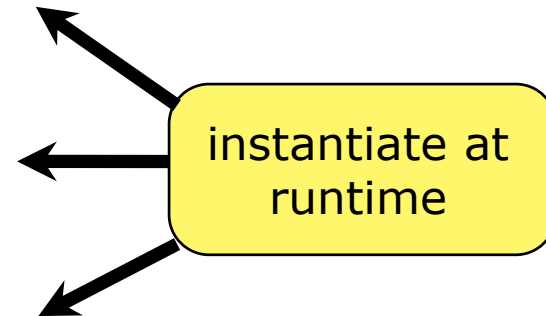
# Solution

Inheritance

# example

Duck `duck`;

```
if (picnic) {  
    duck = new Mallarduck();  
}  
else if (hunting) {  
    duck = new DecoyDuck();  
}  
else if (inBathTub) {  
    duck = new RubberDuck();  
}
```



# Problem

“Too many dependencies to concrete classes makes your software difficult to maintain and modify”

# Adding a new duck

Duck `duck`;

```
if (picnic) {  
    duck = new MallardDuck();  
}  
else if (hunting) {  
    duck = new DecoyDuck();  
}  
else if (inBathTub) {  
    duck = new RubberDuck();  
}  
else if (bill == "blue") {  
    duck = new RuddyDuck();  
}
```

“Too many dependencies to concrete classes makes your software difficult to maintain and modify”

# Dealing with CHANGE

## DESIGN PRINCIPLES

- ☐ Code to an interface and insulate yourself from changes {Strategy}
- ☐ Identify aspects that vary and separate them from what stays the same. {Strategy}
- ☐ Close your code for modification {Decorator}



# Duck Factory

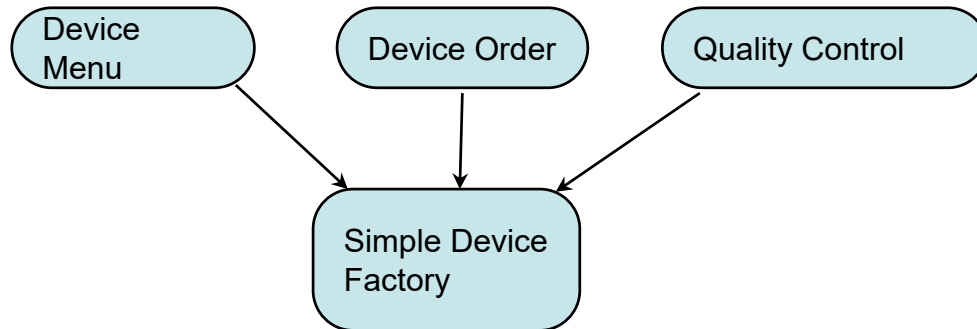
```
public class DuckFactory {  
  
    public Duck createDuck(String type) {  
  
        Duck newDuck;  
        if (type.equals("picnic")){  
            newDuck = new MallardDuck();  
        }  
        else if (type.equals("hunting")){  
            newDuck = new DecoyDuck();  
        }  
        else if (type.equals("bathtub")){  
            newDuck = new RubberDuck();  
        }  
        else if (type.equals("blue bill")){  
            newDuck = new RuddyDuck();  
        }  
        return newDuck;  
    }  
}
```

# Alternate Duck Factory

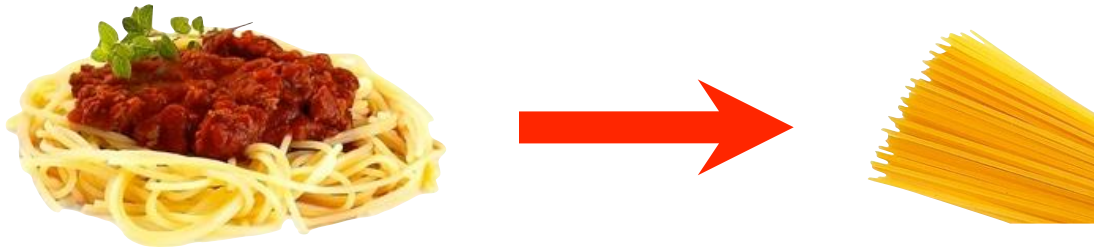
```
public class DuckFactory {  
  
    public Duck createMallardDuck() {  
        return new MallardDuck();  
    }  
    public Duck createDecoyDuck() {  
        return new DecoyDuck();  
    }  
    public Duck createRubberDuck() {  
        return new RubberDuck();  
    }  
    public Duck createRuddyDuck() {  
        return new RuddyDuck();  
    }  
}
```

# Benefits?

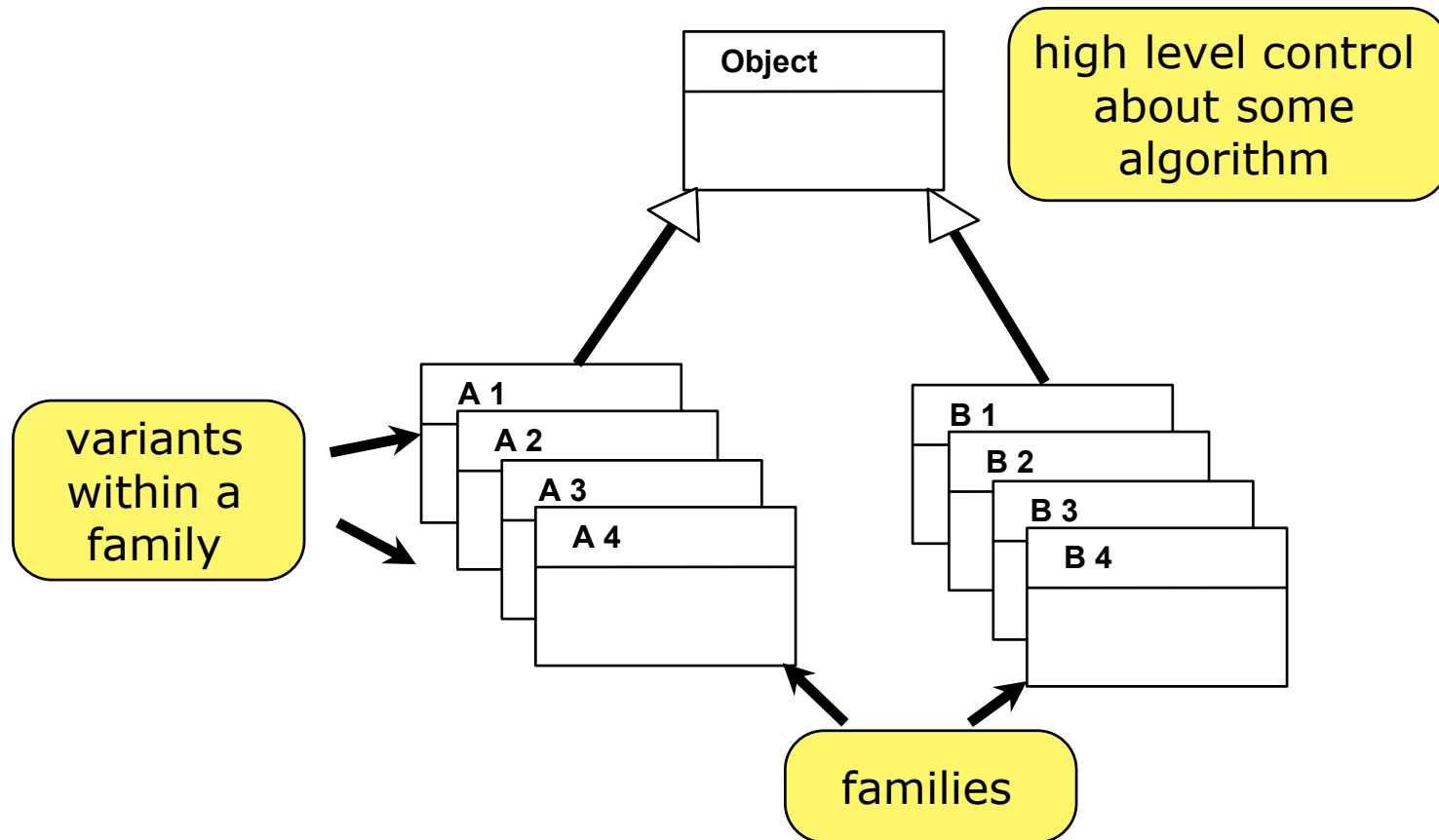
□ put object creation in an object?



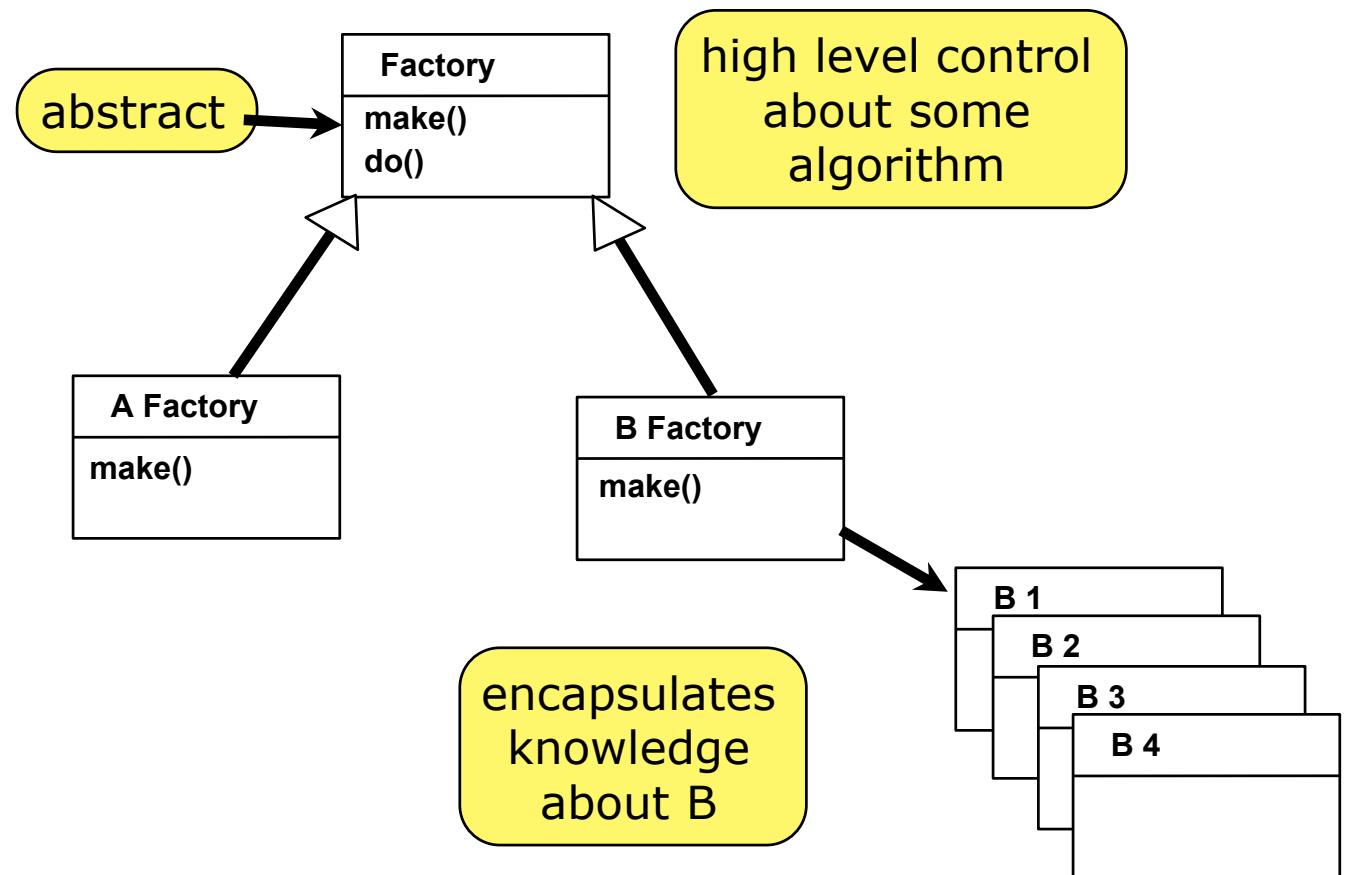
□ "spaghetti-ness" of software



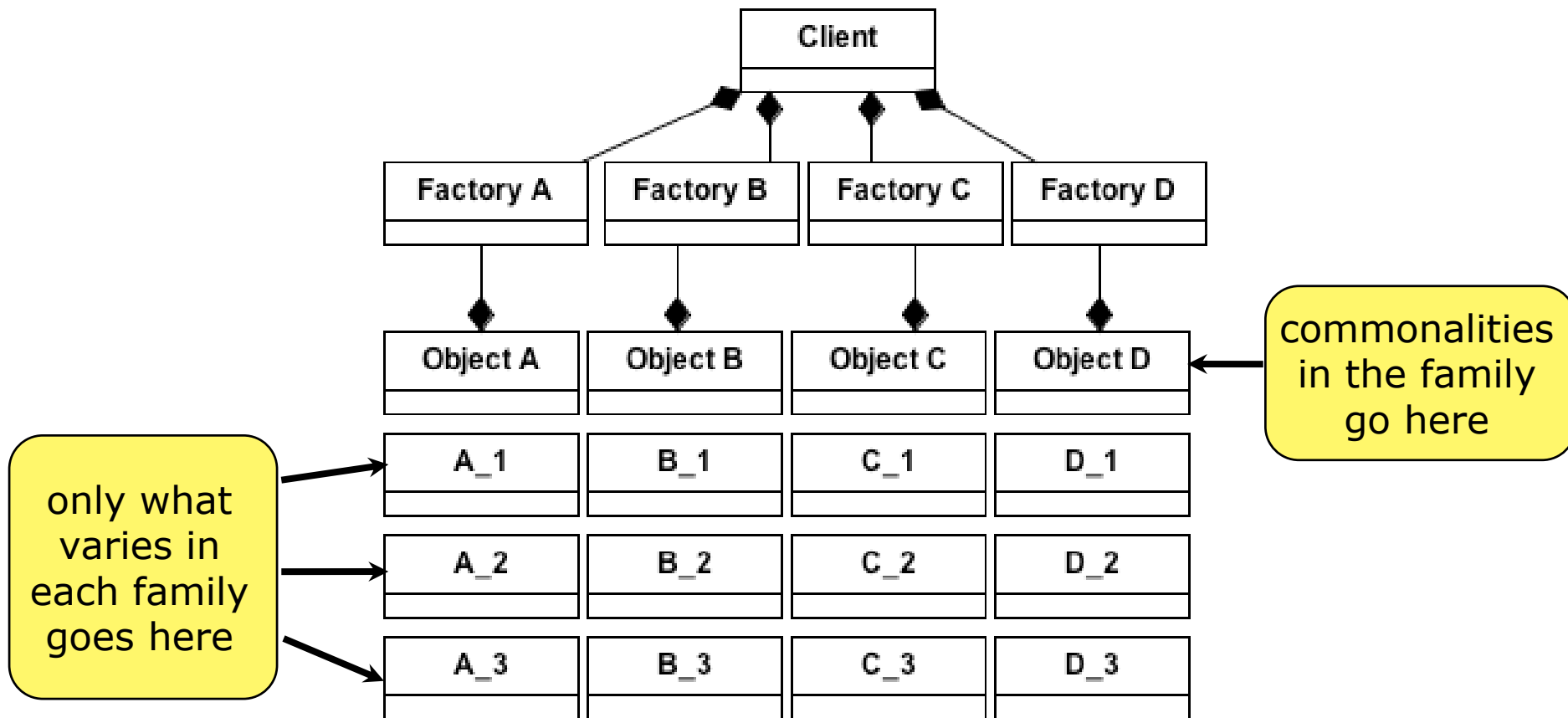
# Product Class



# Creator Class



# benefits



# Factory Method Pattern

The Factory Method Pattern defines an interface for creating an object, but lets subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses

# Factory Method Classes

```
public class AndroidFactory implements DeviceFactory {
```

```
    public Device createMP3() {  
        return new AndroidMP3Player();  
    }
```

```
    public Device createPhone() {  
        return new AndroidPhone();  
    }
```

```
}
```

make DeviceFactory an interface

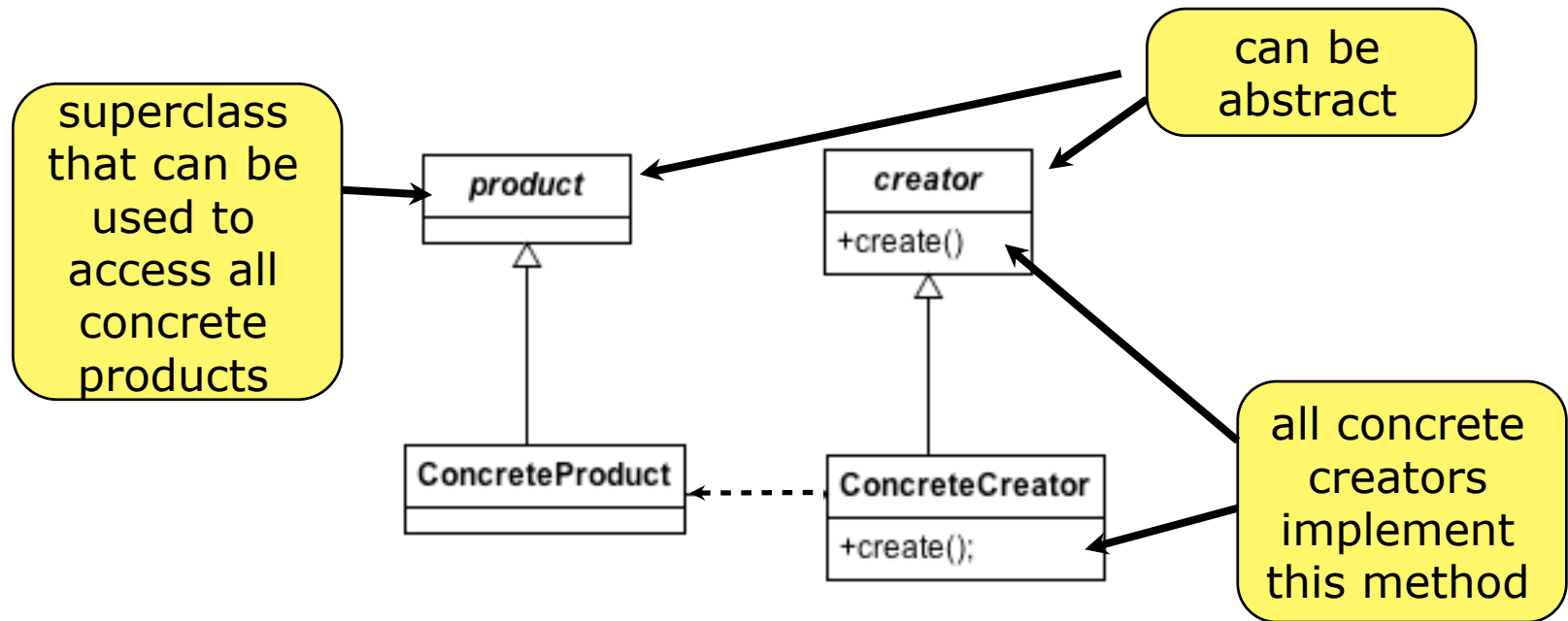
```
public class AndroidMP3 extends MP3Player{  
    private string CPU;
```

```
    public AndroidMP3() {  
        CPU = "Snapdragon";  
    }
```

```
}
```



# Factory Method Diagram



projector

desk

Group 1

Group 2

Group 3

Group 4

Group 5

Group 6

Group 7

Group 8

Group 9

Group 10

Group 11

Group 12

pillar

Group 13

Group 14

Group 15

Group 16

Group 17

Group 18

Group 19

Group 20

Group 21

Group 22

Group 23

Group 24

Group 25

Group 26

Group 27

Group 28

Group 29

pillar

skateboards

door